

ListView i RecyclerView

1. Po co nam listy?

Większość aplikacji pokazuje długie listy podobnych elementów: wiadomości w czacie, posty na Instagramie, piosenki w Spotify, kontakty, produkty w sklepie.

Problem: liczba elementów może być ogromna (setki, tysiące, miliony). Jak je wyświetlić?

Nieudane podejście: osobny widok dla każdego elementu. Przy 10 000 elementów oznacza to:

- ogromne zużycie pamięci,
- bardzo długie uruchamianie ekranu,
- zacinające się przewijanie,
- ryzyko zamknięcia aplikacji przez system.

Rozwiązanie: pokazywać tylko tyle widoków, ile mieści się na ekranie, i używać ich ponownie.

2. ListView

ListView to najstarszy komponent Androida do wyświetlania list, dostępny od pierwszej wersji systemu.

Jak działa:

- wyświetla elementy w jednej kolumnie, jeden pod drugim,
- korzysta z adaptera, który przygotowuje wiersze,
- potrafi używać widoków ponownie, ale programista musi o to sam zadbać.

Ograniczenia:

- tylko pionowa lista (bez siatki i przewijania poziomego),
- recykling jest opcjonalny, więc łatwo napisać wolną listę,
- brak wbudowanych animacji,
- każda zmiana danych odświeża całą listę,
- mała elastyczność wyglądu i zachowania.

Dziś: spotkasz go w starszych aplikacjach i starych tutorialach. Nowe projekty praktycznie go nie używają.

3. RecyclerView

RecyclerView to następca ListView, stworzony, by naprawić jego wady.

Recykling widoków

Ekran: [element 1] ← wyjeżdża za ekran

[element 2]

[element 3]

[element 4]

[element 5] ← wjeżdża: TEN SAM widok co element 1, z nowymi danymi

Jak to działa:

1. Na ekranie mieści się np. 8 elementów, więc powstaje ok. 10 widoków.
2. Użytkownik przewija listę i górny element znika z ekranu.
3. Jego widok nie jest usuwany, tylko trafia do puli recyklingu.
4. Nowy element wjeżdżający na ekran bierze widok z puli.
5. Adapter wpisuje do niego nowe dane.

Wniosek: niezależnie od tego, czy lista ma 50, czy 5 milionów elementów, w pamięci jest tylko kilkanaście widoków.

Części składowe

Część	Rola
Dane	to, co pokazujemy
Adapter	łączy dane z widokami
ViewHolder	trzyma referencje do widoków jednego wiersza
LayoutManager	decyduje, jak elementy są rozmieszczone (lista, siatka)
ItemDecoration	separatory, marginesy, tła (opcjonalnie)
ItemAnimator	animacje dodawania i usuwania (opcjonalnie)

4. Porównanie

Cecha	ListView	RecyclerView
Wiek	stary, od początku Androida	nowszy, następca
Recykling widoków	opcjonalny	wbudowany i wymuszony
Układ	tylko pionowa lista	lista, siatka, mozaika, poziomo
Animacje	brak	wbudowane
Aktualizacja danych	odświeża wszystko	odświeża tylko zmienione elementy
Elastyczność	mała	duża (modularny)
Dziś	stare projekty	standard